

FCFS:

```
//first come first serve
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, j;
```

```
    printf("Enter total number of processes: ");
```

```
    scanf("%d", &n);
```

```
    int at[n], bt[n], ct[n], tat[n], wt[n], pid[n];
```

```
    float avg_tat = 0, avg_wt = 0;
```

```
    // Input Arrival Time and Burst Time
```

```
    for(i = 0; i < n; i++) {
```

```
        pid[i] = i + 1;
```

```
        printf("\nProcess %d\n", pid[i]);
```

```
        printf("Arrival Time: ");
```

```
        scanf("%d", &at[i]);
```

```
        printf("Burst Time: ");
```

```
        scanf("%d", &bt[i]);
```

```
    }
```

```
    // Sort processes by Arrival Time
```

```
    for(i = 0; i < n - 1; i++) {
```

```
        for(j = i + 1; j < n; j++) {
```

```
            if(at[i] > at[j]) {
```

```
int temp;
```

```
temp = at[i];
```

```
at[i] = at[j];
```

```
at[j] = temp;
```

```
temp = bt[i];
```

```
bt[i] = bt[j];
```

```
bt[j] = temp;
```

```
temp = pid[i];
```

```
pid[i] = pid[j];
```

```
pid[j] = temp;
```

```
}
```

```
}
```

```
}
```

```
int current_time = 0;
```

```
// Calculate Completion Time, Turnaround Time, Waiting Time
```

```
for(i = 0; i < n; i++) {
```

```
    if(current_time < at[i])
```

```
        current_time = at[i];
```

```
    ct[i] = current_time + bt[i];
```

```
    tat[i] = ct[i] - at[i];
```

```
    wt[i] = tat[i] - bt[i];
```

```
    current_time = ct[i];
```

```

    avg_tat += tat[i];
    avg_wt += wt[i];
}

avg_tat /= n;
avg_wt /= n;

// Output
printf("\nProcess\tArrival Time\tBurst Time\tCompletion Time\tTurnaround Time\tWaiting
Time\n");

for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
}

printf("\nAverage Turnaround Time: %.2f", avg_tat);
printf("\nAverage Waiting Time: %.2f\n", avg_wt);

return 0;
}

```

OUTPUT:

```
C:\Users\Admin\Desktop\Nitya_OS\fcfs.exe
Enter total number of processes: 3

Process 1
Arrival Time: 0
Burst Time: 10

Process 2
Arrival Time: 1
Burst Time: 5

Process 3
Arrival Time: 0
Burst Time: 2

Process Arrival Time    Burst Time    Completion Time Turnaround Time Waiting Time
P1      0              10              10              10              0
P3      0               2              12              12              10
P2      1               5              17              16              11

Average Turnaround Time: 12.67
Average Waiting Time: 7.00

Process returned 0 (0x0)  execution time : 17.473 s
Press any key to continue.
```

SJF -Non Preemptive:

```
#include <stdio.h>

int main() {
    int n, i, j;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], ct[n], tat[n], wt[n], pid[n];
    int finished[n];

    float avg_tat = 0, avg_wt = 0;

    // Input arrival times
    printf("Enter arrival times:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &at[i]);
        pid[i] = i + 1;
        finished[i] = 0;
    }

    // Input burst times
    printf("Enter burst times:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &bt[i]);
    }

    int completed = 0;
```

```

int current_time = 0;

while(completed < n) {

    int min_bt = 9999;
    int index = -1;

    // Find process with minimum burst time in ready queue
    for(i = 0; i < n; i++) {
        if(at[i] <= current_time && finished[i] == 0) {
            if(bt[i] < min_bt) {
                min_bt = bt[i];
                index = i;
            }
        }
    }

    // If no process is ready
    if(index == -1) {
        current_time++;
    }
    else {
        ct[index] = current_time + bt[index];
        tat[index] = ct[index] - at[index];
        wt[index] = tat[index] - bt[index];

        current_time = ct[index];
        finished[index] = 1;
        completed++;

        avg_tat += tat[index];
    }
}

```

```

        avg_wt += wt[index];
    }
}

avg_tat /= n;
avg_wt /= n;

// Output
printf("\nSJF Scheduling:\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
}

printf("\nAverage turnaround time: %f ms\n", avg_tat);
printf("Average waiting time: %f ms\n", avg_wt);

return 0;
}

```

OUTPUT:

```
C:\Users\Admin\Desktop\Nitya_OS\sjf_nonpreemptive.exe
Enter number of processes: 3
Enter arrival times:
0 0 1
Enter burst times:
12 4 6
SJF Scheduling:
PID    AT    BT    CT    TAT    WT
P1      0    12    22    22    10
P2      0     4     4     4     0
P3      1     6    10     9     3
Average turnaround time: 11.666667 ms
Average waiting time: 4.333333 ms
Process returned 0 (0x0)   execution time : 37.129 s
Press any key to continue.
```


SJF – Preemptive:

```
#include <stdio.h>

int main() {
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], rt[n], ct[n], tat[n], wt[n], pid[n];

    for(i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("\nProcess P%d\n", pid[i]);

        printf("Arrival Time: ");
        scanf("%d", &at[i]);

        printf("Burst Time: ");
        scanf("%d", &bt[i]);

        rt[i] = bt[i]; // Remaining time initially equals burst time
    }

    int completed = 0;
    int current_time = 0;
    int min_index;
    int min_rt;
```

```
float avg_tat = 0, avg_wt = 0;
```

```
while(completed < n) {
```

```
    min_rt = 9999;
```

```
    min_index = -1;
```

```
    // Find process with smallest remaining time
```

```
    for(i = 0; i < n; i++) {
```

```
        if(at[i] <= current_time && rt[i] > 0) {
```

```
            if(rt[i] < min_rt) {
```

```
                min_rt = rt[i];
```

```
                min_index = i;
```

```
            }
```

```
        }
```

```
    }
```

```
    // If no process is ready
```

```
    if(min_index == -1) {
```

```
        current_time++;
```

```
    }
```

```
    else {
```

```
        // Execute process for 1 unit
```

```
        rt[min_index]--;
```

```
        current_time++;
```

```
        // If process finishes
```

```
        if(rt[min_index] == 0) {
```

```
            completed++;
```

```
            ct[min_index] = current_time;
```

```

        tat[min_index] = ct[min_index] - at[min_index];
        wt[min_index] = tat[min_index] - bt[min_index];

        avg_tat += tat[min_index];
        avg_wt += wt[min_index];
    }
}

avg_tat /= n;
avg_wt /= n;

// Output
printf("\nSJF Preemptive (SRTF) Scheduling:\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
}

printf("\nAverage Turnaround Time: %.2f\n", avg_tat);
printf("Average Waiting Time: %.2f\n", avg_wt);

return 0;
}

```

OUTPUT:

```
C:\Users\Admin\Desktop\Nitya_OS\sjf_preemptive.exe
Enter number of processes: 3

Process P1
Arrival Time: 0
Burst Time: 7

Process P2
Arrival Time: 1
Burst Time: 3

Process P3
Arrival Time: 2
Burst Time: 1

SJF Preemptive (SRTF) Scheduling:
PID   AT   BT   CT   TAT  WT
P1     0    7   11   11   4
P2     1    3    5    4    1
P3     2    1    3    1    0

Average Turnaround Time: 5.33
Average Waiting Time: 1.67

Process returned 0 (0x0)   execution time : 20.824 s
Press any key to continue.
```